



```
def deploy_pipeline():
    project = "AI"
    env = "stg-prod-01"

    > DeployResult:
        agent = CodingAgent(model="gpt-4o")
        await agent.analyze_project()
        tests = await agent.run_tests()
        if tests.passed:
            return await agent.deploy(env)

def pipeline = nx.AIPipeline(
    model="Claude-sonnet",
    steps=["analyze", "test", "deploy"],
    max_iterations=50
)

await pipeline.execute()
# Resultado: Sistema completo
# de gestão de pedidos com
# testes unitários e deploy
```

— A PERÍCIA FORENSE DE CONFIGURAÇÕES —

PLAYBOOK — TOKENS SOB PERÍCIA

◆ PARA INICIANTES

16 capítulos

projetos práticos

AUTOR

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

16
CAPÍTULOS

70+
PÁGINAS

∞
PROJETOS

Objetivo do Material

Todo comando, caminho de config ou nome de modelo que chega ate voce e um documento suspeito: pode ter letra timbrada real (a ferramenta existe) e assinatura falsificada (a sintaxe foi inventada). O leitor assume o papel de perito que nunca aceita um documento sem cruzar contra a fonte primaria.

Obra-mãe: otimizacao-tokens-ide-agentica

Passos práticos: 8

Você é: Perito de Configuracao Agentica

Sumário

Objetivo do Material	4
Como usar este playbook	5
Mapa dos Estágios	6
Passos Práticos	7
Passo 1 — O Manual Suspeito: Por Que Comandos Bonitos Enganam	7
Passo 2 — Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repeticao	9
Passo 3 — Modelos Gratuitos Sem Lenda: o Catalogo Real Por Tras do Gateway	11
Passo 4 — Compressao e Cache Semantico Sob a Lupa: Bibliotecas, Nao Comandos	14
Passo 5 — Paralelismo e Resiliencia Que Sempre Funcionaram	17
Passo 6 — Orcamento e Fallback: Medir Antes de Limitar	20
Passo 7 — O Agente Avancado e as Outras Sete Ferramentas: Produto Real, Sintaxe Fabricada	21
Passo 8 — O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo	23
Checklist Mestre	27

Objetivo do Material

Apresentar o caso: um manual de 15 LABs sobre economia de tokens circula com autoridade, mas metade dos comandos nunca funcionaria se copiados e colados. Estabelecer o método do livro (hierarquia de fontes A/B/C, cruzamento contra documentação oficial) e o achado central que orienta tudo: ferramenta real com sintaxe fabricada e mais perigoso que uma fabricação total, porque o nome familiar destrava a confiança do operador.

Como usar este playbook

Você é o **Perito de Configuracao Agentica**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Perícia	1, 2, 3, 4
2	Laudo	5, 6, 7, 8

Passos Práticos

Passo 1 — O Manual Suspeito: Por Que Comandos Bonitos Enganam

Estágio: Perícia · **Origem:** Cap. 1 — O Manual Suspeito: Por Que Comandos Bonitos Enganam

① Objetivo do passo

O leitor entende o padrão de erro central (ferramenta real + sintaxe inventada) e aprende o método de perícia que vai aplicar no resto do livro.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- `settings.json`
- `opencode/<model-id>`

④ Execução

Diagnosticando os dois casos de abertura

```
#!/usr/bin/env bash
# Diagnostico dos dois casos de abertura do Capítulo 1.
# Não depende de rede: usa apenas checagens locais (command -v, teste de path).

echo "=== CASO 1: instalar o ccusage ==="
echo "Comando do manual (fabricado): pipx install ccusage"
if command -v pipx >/dev/null 2>&1; then
    echo "pipx esta presente, mas ccusage e' um pacote Node/npm -- pipx so instala pacotes Python."
    echo "O comando do manual falharia com 'No matching distribution found'."
else
    echo "pipx não está instalado neste ambiente; de qualquer forma o comando falharia (pacote errado)."
fi

echo "Comando real (documentado pelo projeto): npx ccusage@latest daily"
if command -v npx >/dev/null 2>&1; then
    echo "npx disponível -- este e' o caminho real, sem instalacao previa necessaria."
else
    echo "npx não está instalado neste ambiente, mas e' o comando correto segundo a
```

```
documentacao do projeto."
fi

echo ""
echo "=== CASO 2: onde mora a config de cache do Claude Code ==="
CAMINHO_FABRICADO="$HOME/.config/claude-code/settings.json"
CAMINHO_REAL="$HOME/.claude/settings.json"

echo "Caminho citado pelo manual (fabricado): ${CAMINHO_FABRICADO}"
echo "Caminho real documentado pela Anthropic: ${CAMINHO_REAL}"

if [ -f "${CAMINHO_REAL}" ]; then
    echo "Arquivo real encontrado. Campos validos: model, permissions, env, hooks -- nao
cacheControl."
else
    echo "Arquivo real ainda nao existe neste ambiente, mas o caminho e' o unico
documentado oficialmente."
fi
```

Formalizando a árvore de decisão

```
"""
Perito de Configuracao Agentica -- classificador de evidencia (Capitulo 1).

Checklist antes de aceitar qualquer comando/config de um manual de terceiros:
1. Existe documentacao oficial do proprio fornecedor (fonte classe A)?
2. Existe repositorio ou pacote oficial do projeto -- GitHub/PyPI/npm (fonte classe B)?
3. A sintaxe do manual bate exatamente com o que a fonte A ou B documenta?
4. Antes de automatizar, rode "<comando> --help" (ou equivalente) e compare
   com o que o manual afirma -- nunca depois de automatizar.
"""

from dataclasses import dataclass

@dataclass
class Evidencia:
    existe_doc_oficial: bool    # fonte classe A
    existe_repo_pacote: bool    # fonte classe B
    ferramenta_existe: bool    # a ferramenta/produto citado existe de fato
    bate_com_manual: bool      # a sintaxe do manual confere com a fonte encontrada

def classificar_fonte(evidencia: Evidencia) -> str:
    """Aplica a hierarquia de fontes A/B/C e devolve um dos 4 veredictos.

    Regra central deste capitulo: NAO_VERIFICAVEL nunca e' sinonimo de
    aprovado. Sem fonte primaria (classe A ou B), o comando fica pendente
    ate confirmacao manual -- nunca entra em producao so porque "parece
    plausivel".
    """
    tem_fonte_primaria = evidencia.existe_doc_oficial or evidencia.existe_repo_pacote

    if not tem_fonte_primaria:
        if evidencia.ferramenta_existe:
            return "NAO_VERIFICAVEL"
        return "FABRICADO"
```

```

    return "CONFIRMADO" if evidencia.bate_com_manual else "PARCIALMENTE_CORRETO"

if __name__ == "__main__":
    casos = {
        "p

```

Registrando a classe da fonte no laudo

```

def classe_predominante(evidencia: Evidencia) -> str:
    """Retorna o rótulo de classe (A/B/C) que sustenta o veredicto.

    Uma fonte classe C (blog, guia de comunidade) nunca aparece aqui como
    suficiente sozinha -- ela só recebe o rótulo "C" quando nem A nem B
    existem, para deixar explícito no laudo que aquela evidência é
    corroborativa, nunca primária.
    """
    if evidencia.existe_doc_oficial:
        return "A"
    if evidencia.existe_repo_pacote:
        return "B"
    return "C" if evidencia.ferramenta_existe else "sem_fonte"

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Escolha 1 comando de qualquer manual, tutorial ou post de blog que você usou nas últimas semanas sem checar a fonte oficial
- ☐ Rode `<comando> --help` (ou o equivalente da ferramenta) e compare linha a linha com o que você havia copiado
- ☐ Classifique o comando usando os 4 veredictos deste capítulo: CONFIRMADO, PARCIALMENTE CORRETO, FABRICADO ou NÃO VERIFICÁVEL
- ☐ Se for PARCIALMENTE CORRETO ou FABRICADO, escreva a versão corrigida ao lado da original, com a fonte primária anotada

⑦ Armadilhas

- (a completar)

Passo 2 — Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repetição

Estágio: Perícia · **Origem:** Cap. 2 — Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repetição

① Objetivo do passo

O leitor entende o mecanismo real de cache de prompt (desconto de até 90% em leitura) e onde ele de fato se configura, sem o caminho de arquivo fabricado do manual original.

② Pré-requisito

Passo 1 concluído

③ Entregas

- `~/.config/claude-code/settings.json`
- `~/.claude/settings.json`
- `.claude/settings.json`
- `.claude/settings.local.json`
- `CLAUDE.md`
- `~/.aider.conf.yml`

④ Execução

Onde a configuração de verdade mora, por ferramenta

```
{
  "model": "claude-sonnet-4-5",
  "env": {
    "ANTHROPIC_MODEL": "claude-sonnet-4-5"
  },
  "permissions": {
    "allow": ["Bash(git log:*)", "Read(**)"]
  },
  "cleanupPeriodDays": 30
}
```

Verificando a estabilidade do prefixo com sha256sum

```
# Grava o hash atual do CLAUDE.md e compara com o hash da execucao anterior.
# Se os hashes diferem, o prefixo mudou e o cache sera reescrito na proxima chamada.
ARQUIVO="CLAUDE.md"
HASH_ANTERIOR_ARQ=".claude_md.sha256"

HASH_ATUAL=$(sha256sum "$ARQUIVO" | cut -d ' ' -f1)

if [ -f "$HASH_ANTERIOR_ARQ" ]; then
  HASH_ANTERIOR=$(cat "$HASH_ANTERIOR_ARQ")
  if [ "$HASH_ATUAL" = "$HASH_ANTERIOR" ]; then
    echo "CLAUDE.md estavel -- prefixo intacto, cache elegivel para READ."
  else
    echo "CLAUDE.md mudou desde a ultima execucao -- proxima chamada sera WRITE."
  fi
else
  echo "Primeira execucao registrada -- sem baseline para comparar ainda."
fi

echo "$HASH_ATUAL" > "$HASH_ANTERIOR_ARQ"
```

Medindo cache hit de verdade

```
# instalacao/uso real: sem instalar nada, via npx  
npx ccusage@latest session --json
```

⑤ Verificação / Gate

```
npx ccusage@latest session --json
```

⑥ Feito quando...

- ☐ Localize o arquivo real de configuração do Claude Code na sua máquina (`~/.claude/settings.json`) e liste os campos existentes — confirme que não há `cacheControl` nem `systemPrompt`
- ☐ Rode `npx ccusage@latest session --json` em um projeto com Claude Code já usado e identifique os valores de `cacheCreationTokens`, `cacheReadTokens` e `cacheHitRate`
- ☐ Se você usa Aider, adicione `cache-prompts: true` ao `~/.aider.conf.yml` e rode duas chamadas seguidas sobre o mesmo arquivo para observar a diferença de custo
- ☐ Escreva, para o seu próprio `CLAUDE.md`, uma lista dos elementos que poderiam variar de uma execução para outra (data, hora, IDs aleatórios) e remova qualquer um deles do topo do arquivo

⑦ Armadilhas

- (a completar)

Passo 3 — Modelos Gratuitos Sem Lenda: o Catalogo Real Por Tras do Gateway

Estágio: Perícia · **Origem:** Cap. 3 — Modelos Gratuitos Sem Lenda: o Catalogo Real Por Tras do Gateway

① Objetivo do passo

O leitor entende que o gateway de modelos gratuitos existe de fato, mas que nomes de modelo e tabelas de especificacao precisam ser conferidos no catalogo vigente, nunca copiados de um manual estatico.

② Pré-requisito

Passo 2 concluído

③ Entregas

- `opencode.json`
- `~/.config/opencode/opencode.json`
- `./opencode.json`

- `provider/model-id`
- `~/.aider.conf.yml`
- `~/.codex/config.toml`

④ Execução

A configuração real: `opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/big-pickle",
  "agent": {
    "build": {
      "model": "opencode/grok-code-fast-1"
    }
  }
}
```

O erro que você não vai cometer: config fabricada vs. config real

```
# ERRADO — nome de modelo fabricado, imitando o padrao de "porte" do manual auditado.
# opencode.json nao tem campo defaultModel nem cacheControl no nivel raiz.
cat > opencode-errado.json <<'EOF'
{
  "defaultModel": "opencode-zen-free-medium",
  "cacheControl": { "type": "ephemeral" },
  "maxTokens": 4096
}
EOF

# CORRETO — campo real "model", formato provider/model-id confirmado na doc oficial.
cat > opencode-correto.json <<'EOF'
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/big-pickle"
}
EOF

echo "Config fabricada gravada em opencode-errado.json (referencia de erro, nao use)."
```

Detectando quando o catálogo mudou

```
#!/usr/bin/env bash
# verificar-catalogo.sh — sinaliza quando a pagina de doc do catalogo mudou
# desde a ultima checagem, para voce reconferir o nome do modelo antes de usar.
set -euo pipefail

URL="https://opencode.ai/docs/zen/"
CACHE_DIR="$HOME/.cache/pericia-catalogo"
HASH_FILE="$CACHE_DIR/opencode-zen.sha256"

mkdir -p "$CACHE_DIR"

hash_atual=$(curl -fsSL "$URL" | sha256sum | awk '{print $1}')

if [[ -f "$HASH_FILE" ]]; then
```

```

hash_anterior=$(cat "$HASH_FILE")
if [[ "$hash_atual" != "$hash_anterior" ]]; then
    echo "ALERTA: o catalogo do OpenCode Zen mudou desde a ultima checagem."
    echo "Reconfira os model-id antes de publicar ou automatizar."
else
    echo "Catalogo sem mudanca detectada desde a ultima checagem."
fi
else
    echo "Primeira checagem registrada."
fi

echo "$hash_atual" > "$HASH_FILE"

```

Bônus: o script de benchmark que já estava certo

```

#!/usr/bin/env bash
# benchmark-model.sh -- mede a latencia real de uma chamada a um model-id do
# catalogo Zen. A tecnica de cronometragem (date +%s%N, nanossegundos) e
# generica e valida -- o erro do manual auditado nunca foi o cronometro,
# foi aplicar essa tecnica a nomes de modelo que nao existem [2][3].
set -euo pipefail

MODELO="$1:informe um model-id conferido em opencode.ai/docs/zen/" # ex.: opencode/
big-pickle [1]
PROMPT="$2:-Explique em uma frase o que e cache semantico.}"

chamada_ao_modelo() {
    # Substitua esta funcao pela chamada real ao SDK ou ao endpoint HTTP
    # autenticado do seu gateway -- este script nao inventa uma sintaxe de CLI
    # que o OpenCode nao documenta.
    echo "[integre aqui a chamada real ao modelo $MODELO]"
}

inicio_ns=$(date +%s%N)
resposta=$(chamada_ao_modelo)
fim_ns=$(date +%s%N)

latencia_ms=$(( (fim_ns - inicio_ns) / 1000000 ))
echo "Modelo testado: $MODELO"
echo "Latencia medida: ${latencia_ms}ms"
echo "Resposta: $resposta"

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Confiar em tabelas de benchmark encontradas em posts sem link para a medição original — trate como não verificado até achar a fonte primária
- ☐ Presumir que uma convenção de nome “bonita” (tiny/small/medium) é sinal de autenticidade — muitas vezes é o oposto
- ☐ Automatizar a escolha de modelo sem registrar a data da última checagem de catálogo — sem isso, você não sabe quando reconfirir

⑦ Armadilhas

- (a completar)

Passo 4 — Compressao e Cache Semantico Sob a Lupa: Bibliotecas, Nao Comandos

Estágio: Perícia · **Origem:** Cap. 4 — Compressao e Cache Semantico Sob a Lupa: Bibliotecas, Nao Comandos

① Objetivo do passo

O leitor aprende a diferenca entre uma biblioteca Python real e uma interface de linha de comando inventada, usando as duas ferramentas de compressao/cache semantico do manual como caso pratico.

② Pré-requisito

Passo 3 concluído

③ Entregas

- `__main__.py`
- `~/.gptcache/config.yaml`
- `cache.init(...)`
- `/put`
- `/get`
- `semantic_cache.py`

④ Execução

A Classe Certa: PromptCompressor em Vez de Llmlingua

```
#!/usr/bin/env python3
"""compress_prompt.py -- wrapper real de compressao de prompt via Llmlingua.

O manual descreve `Llmlingua().compress_prompt(prompt, preservation_rate=0.5)`
e uma CLI `python -m llmlingua --preserve`. Nenhuma das duas existe: a classe
real e o parametro real sao outros -- ver [5][6].

Uso:
    python compress_prompt.py --rate 0.5 < prompt_bruto.txt > prompt_comprimido.txt
Requisito:
    pip install llmlingua
"""
import argparse
import sys

from llmlingua import PromptCompressor # classe real; nao "Llmlingua"
```

```
def comprimir(texto: str, rate: float, contexto_longo: bool = False) -> dict:
    """Comprime um prompt usando a API real do LLMlingua."""
    compressor = PromptCompressor()
    kwargs = {"rate": rate}
    if contexto_longo:
        # LongLLMLingua não é pacote separado: é este parâmetro do método real [5]
        kwargs["rank_method"] = "longllmlingua"
    return compressor.compress_prompt(texto, **kwargs)

def main() -> int:
    parser = argparse.ArgumentParser(
        description="Comprime um prompt via LLMlingua (API real, sem CLI oficial)."
    )
    parser.add_argument("--rate", type=float, default=0.5,
                        help="fracção de tokens a preservar, 0 a 1")
    parser.add_argument("--contexto-longo", action="store_true",
                        help="ativa o modo longllmlingua para prompts extensos")
    args = parser.parse_args()

    bruto = sys.stdin.read()
    if not bruto.strip():
        print("erro: nenhum prompt recebido via stdin", file=sys.stderr)
        return 1

    resultado = comprimir(bruto, args.rate, ar
```

O Hook de Cache Semântico Completo (semantic_cache.py)

```
"""semantic_cache.py -- hook real de cache semântico via GPTCache.

O manual descreve `gptcache init/list/add/query/stats/clear` como CLI e um
arquivo `~/.gptcache/config.yaml`. Nenhum dos dois existe: a configuração do
GPTCache é inteiramente programática [7].
"""

from gptcache import cache
from gptcache.manager import get_data_manager, CacheBase, VectorBase
from gptcache.embedding import Onnx
from gptcache.similarity_evaluation.distance import SearchDistanceEvaluation
from gptcache.adapter.api import get, put

_onnx = Onnx()
_data_manager = get_data_manager(
    CacheBase("sqlite"),
    VectorBase("faiss", dimension=_onnx.dimension),
)

def inicializar(limiar_similaridade: float = 0.8) -> None:
    """Substitui o YAML fabricado por chamadas reais de inicialização [7]."""
    cache.init(
        embedding_func=_onnx.to_embeddings,
        data_manager=_data_manager,
        similarity_evaluation=SearchDistanceEvaluation(),
    )
    cache.config.similarity_threshold = limiar_similaridade
```

```
def consultar_ou gerar(pergunta: str, gerar_resposta) -> str:
    """Consulta o cache semantico; so chama o modelo se nao houver hit."""
    resposta_em_cache = get(pergunta)
    if resposta_em_cache is not None:
        return resposta_em_cache

    resposta = gerar_resposta(pergunta)
    put(pergunta, resposta)
    return resposta

if __name__ == "__main__":
    inicializar(limiar_similaridade=0.8)

    def chamada_simulada(pergunta: str) -> str:
        return f"resposta gerada para: {pergunta}"

    print(consultar_ou gerar("como configurar cache de prompt no claude code", chamada_
```

A Segunda Forma Real de Configurar o Limiar: Objeto Config

```
from gptcache import cache
from gptcache.config import Config

cache.init(
    embedding_func=_onnx.to_embeddings,
    data_manager=_data_manager,
    similarity_evaluation=SearchDistanceEvaluation(),
    config=Config(similarity_threshold=0.8), # forma alternativa, mesmo efeito
)
```

O Modo Servidor: gptcache_server, Para Quando Você Não Quer Embutir o Cache no Script

```
# unico binario real de linha de comando do projeto -- nao existem os
# subcomandos init/list/add/query/stats/clear que o manual descreve [7].
gptcache_server -s 127.0.0.1 -p 8000
```

⑤ Verificação / Gate

```
python compress_prompt.py --rate 0.5 < prompt_bruto.txt > prompt_comprimido.txt
```

⑥ Feito quando...

- ☐ Rode `pip show llmlingua` no seu ambiente e confirme que não aparece nenhum “Console Scripts” na saída
- ☐ Escreva e teste `compress_prompt.py` com um prompt real de pelo menos 500 tokens, comparando o texto antes e depois
- ☐ Inicialize `cache.init()` do GPTCache localmente e force um cache hit repetindo a mesma pergunta duas vezes seguidas
- ☐ Ajuste `similarity_threshold` para 0.9 e depois para 0.6 e registre a diferença de comportamento em duas perguntas parecidas, mas não idênticas

- ☐ Suba `gptcache_server -s 127.0.0.1 -p 8000` localmente e confirme, consultando a doc oficial [7], o schema de payload real dos endpoints `/put` e `/get` antes de integrá-lo a qualquer script

⑦ Armadilhas

- (a completar)

Passo 5 — Paralelismo e Resiliencia Que Sempre Funcionaram

Estágio: Laudo · **Origem:** Cap. 5 — Paralelismo e Resiliencia Que Sempre Funcionaram

① Objetivo do passo

O leitor implementa, com confiança total, os padroes de engenharia do manual que passaram na perícia sem ressalva: paralelismo controlado, circuit breaker e backoff exponencial.

② Pré-requisito

Passo 4 concluído

③ Entregas

- `CLAUDE.md`
- `settings.json`
- `.aider.conf.yml`
- `config.toml`

④ Execução

Paralelismo Controlado em Python: semáforo Antes de Disparar Tudo

```
import asyncio
import random

async def revisar_arquivo(caminho: str, semaforo: asyncio.Semaphore) -> dict:
    """Simula uma chamada de agente revisando 1 arquivo, respeitando o teto de
    concorrencia."""
    async with semaforo:
        # Ponto onde entraria a chamada real ao modelo/API do agente.
        await asyncio.sleep(random.uniform(0.01, 0.05))
        return {"arquivo": caminho, "status": "revisado"}

async def revisar_lote(arquivos: list[str], concorrencia_maxima: int = 5) -> list[dict]:
    semaforo = asyncio.Semaphore(concorrencia_maxima)
    tarefas = [revisar_arquivo(c, semaforo) for c in arquivos]
    return await asyncio.gather(*tarefas)

if __name__ == "__main__":
```

```
arquivos = [f"modulo_{i}.py" for i in range(12)]
resultados = asyncio.run(revisar_lote(arquivos, concorrencia_maxima=5))
print(f"{len(resultados)} arquivos revisados, teto de 5 simultaneos")
```

O Disjuntor Que Já Existia: Circuit Breaker Testável

```
import time
from enum import Enum

class EstadoCircuito(Enum):
    FECHADO = "fechado"
    ABERTO = "aberto"
    SEMI_ABERTO = "semi_aberto"

class CircuitBreaker:
    def __init__(self, limite_falhas: int = 3, tempo_espera_s: float = 30.0):
        self.limite_falhas = limite_falhas
        self.tempo_espera_s = tempo_espera_s
        self.falhas_consecutivas = 0
        self.estado = EstadoCircuito.FECHADO
        self.momento_abertura = None

    def _pode_tentar(self) -> bool:
        if self.estado == EstadoCircuito.FECHADO:
            return True
        if self.estado == EstadoCircuito.ABERTO:
            expirou = (time.monotonic() - self.momento_abertura) >= self.tempo_espera_s
            if expirou:
                self.estado = EstadoCircuito.SEMI_ABERTO
                return True
            return False
        return True # SEMI_ABERTO: permite a chamada de teste

    def chamar(self, funcao, *args, **kwargs):
        if not self._pode_tentar():
            raise RuntimeError(f"circuito {self.estado.value}: chamada recusada sem tentar rede")
        try:
            resultado = funcao(*args, **kwargs)
        except Exception:
            self.falhas_consecutivas += 1
            if self.falhas_consecutivas >= self.limite_falhas:
                self.estado = EstadoCircuito.ABERTO
                self.momento_abertura = time.monotonic()
            raise
        else:
            self.falhas_consecutivas = 0
            self.estado = EstadoCircuito.FECHADO
            return resultado

if __name__ == "__main__":
    disjuntor = CircuitBreaker(limite_falhas
```

Backoff Exponencial Com Jitter: Espaçando as Novas Tentativas

```
import random
import time
```

```
def com_backoff_jitter(funcao, tentativas_max: int = 5, base_s: float = 1.0):
    """Executa 'funcao' com backoff exponencial (1s, 2s, 4s...) mais jitter de ate
    0.5s."""
    for tentativa in range(tentativas_max):
        try:
            return funcao()
        except Exception as erro:
            if tentativa == tentativas_max - 1:
                raise
            espera = (base_s * (2 ** tentativa)) + random.uniform(0, 0.5)
            print(f"tentativa {tentativa + 1} falhou ({erro}); nova tentativa em
{espera:.2f}s")
            time.sleep(min(espera, 0.01)) # tempo reduzido aqui so para o smoke test do
livro

if __name__ == "__main__":
    contador = {"chamadas": 0}

    def chamada_com_falha_temporaria():
        contador["chamadas"] += 1
        if contador["chamadas"] < 3:
            raise TimeoutError("rate limit do provedor")
        return "resposta do modelo"

    resultado = com_backoff_jitter(chamada_com_falha_temporaria, tentativas_max=5,
base_s=1.0)
    assert resultado == "resposta do modelo"
    assert contador["chamadas"] == 3
    print(f"sucesso na tentativa {contador['chamadas']} apos backoff com jitter [6]")
```

Combinando os Três Padrões em Uma Única Chamada

```
# Reaproveita CircuitBreaker e com_backoff_jitter definidos nos blocos anteriores
# desta mesma secao – este trecho ilustra a composicao, nao roda isolado.

def chamar_agente_protegido(prompt: str, disjuntor: CircuitBreaker, tentativas_max: int =
3) -> str:
    """Combina circuit breaker + backoff com jitter numa unica chamada resiliente.
    O teto de concorrencia fica uma camada acima (Semaphore/xargs -P/parallel -j),
    decidindo quantas chamadas como esta podem estar em voo ao mesmo tempo."""
    def chamada_real():
        return disjuntor.chamar(lambda: f"resposta para: {prompt}")
    return com_backoff_jitter(chamada_real, tentativas_max=tentativas_max, base_s=0.01)

if __name__ == "__main__":
    disjuntor = CircuitBreaker(limite_falhas=3, tempo_espera_s=5.0)
    resposta = chamar_agente_protegido("resuma o arquivo X", disjuntor)
    assert resposta.startswith("resposta para:")
    print(f"chamada protegida nas tres camadas: {resposta}")
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Imagine a cena: você acabou de terminar o Capítulo 4
- ☐ Você escreve rápido um laço com `asyncio.gather` chamando as 80 revisões de uma vez
- ☐ Você roda o script e
- ☐ O diagnóstico é o mesmo problema que a seção Explica descreveu: `asyncio.gather` sozinho não impõe limite de concorrência nenhum — ele dispara todas as corrotinas fornecidas simultaneamente
- ☐ A correção tem duas partes: primeiro
- ☐ Meça a taxa de erro com os subcomandos reais de auditoria de uso que o Capítulo 6 destrincha a partir da ferramenta de contagem de tokens [21]
- ☐ Como síntese rápida das armadilhas mais comuns nesta frente: (1) esquecer o semáforo

⑦ Armadilhas

- (a completar)

Passo 6 — Orcamento e Fallback: Medir Antes de Limitar

Estágio: Laudo · **Origem:** Cap. 6 — Orcamento e Fallback: Medir Antes de Limitar

① Objetivo do passo

O leitor monta um sistema de controle de gasto e um fallback para modelo local usando os subcomandos e nomes de modelo reais, corrigindo os que o manual errou.

② Pré-requisito

Passo 5 concluído

③ Entregas

- `~/.aider.conf.yml`
- `~/.codex/config.toml`
- `opencode.json`
- `.jsonc`
- `opencode/<model-id>`
- `opencode-zen-free-tiny/small/...`

④ Execução

Alternativa ao Cron: Acoplado o Guard a um Hook do Claude Code

```
# crontab -e — checa o gasto do dia a cada hora, das 8h as 20h.
0 8-20 * * * /caminho/para/token-guard.sh "verificacao de rotina" >> ~/.token-guard-
cron.log 2>&1
```

⑤ Verificação / Gate

```
npx ccusage@latest session --json | jq -r '.sessions[-1] // {}'
```

⑥ Feito quando...

- ☐ Imagine a cena: são 16h de uma sexta-feira
- ☐ O comando `pipx install ccusage` falha silenciosamente para você — na verdade
- ☐ Você gasta vinte minutos reinstalando `pipx`
- ☐ O diagnóstico correto seria outro: `ccusage` nunca foi um pacote Python
- ☐ É uma ferramenta Node — o próprio nome do ecossistema já era a evidência que faltava conferir antes de tentar instalar
- ☐ A correção é trivial depois que o laudo é refeito: `npx ccusage@latest daily --json` roda sem instalação alguma
- ☐ O erro comum não foi de sintaxe de shell — foi de não confirmar

⑦ Armadilhas

- (a completar)

Passo 7 — O Agente Avançado e as Outras Sete Ferramentas: Produto Real, Sintaxe Fabricada

Estágio: Laudo · **Origem:** Cap. 7 — O Agente Avançado e as Outras Sete Ferramentas: Produto Real, Sintaxe Fabricada

① Objetivo do passo

O leitor aplica a perícia em um caso denso: um agente de automação real cujos comandos de skill, cron, memória e sessão foram todos documentados de memória, mais um panorama das outras sete ferramentas citadas no manual original.

② Pré-requisito

Passo 6 concluído

③ Entregas

- `hermes_pericia.sh`
- `~/.mimo/config.yaml`
- `mimocode.json`
- `~/.antigravity/settings.json`
- `~/.google-ai/configurerc`
- `~/.gemini/settings.json`

④ Execução

Checklist de Terminal: `hermes_pericia.sh`

```
#!/usr/bin/env bash
# hermes_pericia.sh -- checklist de pericia dos comandos do Hermes Agent.
#
# O manual de referencia descreve `hermes skill activate`, `hermes cron create`
# --name/--schedule/--deliver`, `hermes memory add --target/--content` e
# `hermes session search/read`. Nenhum dos quatro existe na CLI real -- ver
# a documentacao oficial de CLI/Configuration da NousResearch [4][5].
#
# Uso: ./hermes_pericia.sh (sem argumentos -- imprime o laudo comparativo)

set -euo pipefail

echo "== LAUDO PERICIAL: Hermes Agent =="
echo

printf "%-45s | %-45s\n" "FABRICADO (manual)" "REAL (doc oficial)"
printf "%-45s+%-45s\n" "${printf -- '-%.0s' {1..45}}" "${printf -- '-%.0s' {1..45}}"
printf "%-45s | %-45s\n" "hermes skill activate <nome>" "hermes skills install
<id> | browse | list"
printf "%-45s | %-45s\n" "hermes cron create --name --schedule" "hermes cron create
\"<prompt>\" [--skill NOME]"
printf "%-45s | %-45s\n" "hermes memory add --target --content" "hermes memory setup |
status | off"
printf "%-45s | %-45s\n" "hermes session search --query" "hermes sessions list |
browse | export"
printf "%-45s | %-45s\n" "hermes session read --session-id" "hermes --resume <id>
(ou -r)"
echo
echo "Confirmados sem correcao: hermes cron list | hermes cron remove <id>"
echo "Config real (caminho confere): ~/.hermes/config.yaml"
echo " campos reais: model, terminal, memory, skills, agent, prompt_caching, runtime,
worktree"
echo " campos fabricados: system_prompt, cache_control.type"
```

Regra Geral: `verificar_cli` Antes de Automatizar

```
#!/usr/bin/env python3
"""verificar_cli.py -- protocolo minimo de pericia para qualquer CLI nova.

Regra do capitulo: rodar --help ANTES de automatizar, nunca depois. Este
script generico roda `<binario> --help` (ou `<binario> <subcomando> --help`)
e verifica se os comandos que voce pretende automatizar aparecem de fato na
saida oficial -- a mesma verificacao que teria barrado `hermes skill activate`
antes de ir parar em um script de producao.

Uso:
    python verificar_cli.py hermes --comandos-esperados "skills,cron,memory,sessions"
"""

import argparse
import subprocess
import sys

def rodar_help(binario: str, subcomando: str | None = None) -> str:
    """Executa `<binario> [subcomando] --help` e devolve a saida combinada."""
    comando = [binario] + ([subcomando] if subcomando else []) + ["--help"]
    resultado = subprocess.run(
        comando, capture_output=True, text=True, timeout=15, check=False
```

```
)
return (resultado.stdout or "") + (resultado.stderr or "")

def verificar(binario: str, comandos_esperados: list[str]) -> dict:
    """Confere quais comandos esperados aparecem na saída real de --help."""
    try:
        saida = rodar_help(binario)
    except (FileNotFoundError, subprocess.TimeoutExpired) as erro:
        return {"erro": f"não foi possível rodar '{binario} --help': {erro}"}

    saida_lower = saida.lower()
    confirmados = [c for c in comandos_esperados if c.lower() in saida_lower]
    nao_confirmados = [c for c in comandos_esperados if c.lower() not in saida_lower]
    return {
        "binario": binario,
        "confirmados_na_saida_do_help": confirmados,
        "nao_confirmados_na_saida_do_help": nao_confirmados
    }
```

⑤ Verificação / Gate

```
python verificar_cli.py hermes --comandos-esperados "skills,cron,memory,sessions"
```

⑥ Feito quando...

- ❑ Rode `hermes skills --help`, `hermes cron --help`, `hermes memory --help` e `hermes sessions --help` na sua instalação e confira se os subcomandos batem com o laudo deste capítulo
- ❑ Execute `hermes_pericia.sh` e adicione uma linha nova para o próximo comando do Hermes que você pretende automatizar, antes de escrevê-lo em produção
- ❑ Escolha uma das sete ferramentas da tabela de campo (a que você já usa ou pretende testar) e confirme o caminho real de configuração na documentação oficial dela, registrando o resultado
- ❑ Rode `verificar_cli.py` contra uma CLI Agêntica qualquer que você já tenha instalada, com uma lista de 3 a 5 subcomandos que você pretende automatizar

⑦ Armadilhas

- (a completar)

Passo 8 — O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo

Estágio: Laudo · **Origem:** Cap. 8 — O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo

① Objetivo do passo

O leitor consolida um protocolo pessoal de verificacao reutilizavel, versiona suas configuracoes confirmadas e se ve, ao fechar o livro, como o profissional que audita ferramentas de IA em vez de apenas copia-las.

② Pré-requisito

Passo 7 concluído

③ Entregas

- CLAUDE.md
- ~/.config/<produto>/
- auditar-comando.sh

④ Execução

O Protocolo de perícia em Formato Portátil

```
# protocolo-pericia.yaml
# Preencha um bloco destes por comando suspeito de um manual/tutorial novo.
comando_sob_suspeita: "chezmoi commit -m \"atualiza dotfiles\""
produto_citado: "chezmoi"
fonte_classificada:
  classe: "A"          # A = doc oficial | B = repo/pacote oficial | C = terceiro
  origem: "https://www.chezmoi.io/install/"
sinais_de_alerta:
  - "subcomando nao aparece na lista de comandos do --help oficial"
  - "nome do produto e real, o que reduz a desconfianca natural do operador"
contraprova_realizada: true
veredito: "rejeitado"  # aprovado | pendente | rejeitado
correcao_aplicada: "chezmoi cd && git commit -m \"atualiza dotfiles\""
observacao: "chezmoi nao tem subcomando commit nativo; commit acontece no repositorio git do diretorio-fonte"
```

Restaurando o Cofre Confirmado com chezmoi (Sintaxe Real)

```
$ sh -c "$(curl -fsLS https://get.chezmoi.io)"
chezmoi: instalado em ~/.local/bin/chezmoi

$ chezmoi init
chezmoi: repositorio fonte inicializado em ~/.local/share/chezmoi

$ chezmoi add ~/.claude/CLAUDE.md
$ chezmoi add ~/.claude/settings.json

$ chezmoi status
A  .claude/CLAUDE.md
A  .claude/settings.json

$ sha256sum ~/.claude/CLAUDE.md
a3f2c9e7d1b04f6c8a2e9d5b7c1f0a4e6d8b2c4f  /home/voce/.claude/CLAUDE.md

$ chezmoi cd
$ git commit -am "cofre confirmado: CLAUDE.md e settings.json do Capitulo 8"
$ exit
```

```
$ chezmoi git commit -- -m "alternativa sem sair do diretorio fonte, mesmo efeito"

$ op signin
Digite sua senha mestra do 1Password: *****
Sessão autenticada.

$ op item create --title "API Key" --vault Private --generate-password
Criado item "API Key" no cofre "Private".

$ op read "op://Private/API Key/credential"
sk-ant-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

O Script de Alerta Rápido

```
#!/usr/bin/env bash
# auditar-comando.sh – triagem heurística de comandos suspeitos em um manual novo
# Uso: auditar-comando.sh < trecho-do-manual.txt

set -euo pipefail

PADROES_SUSPEITOS=(
  '--target [a-z]+ --content'
  'skill activate'
  '~/\.config/[a-z-]+/config\.(json|yaml)'
  'commit -m'
  'session (search|read) --'
)

echo "== Triagem heurística (Capítulo 8) =="
while IFS= read -r linha; do
  for padrao in "${PADROES_SUSPEITOS[@]}; do
    if echo "$linha" | grep -qE "$padrao"; then
      echo "SUSPEITO: $linha"
      echo "  -> classifique a fonte (A/B/C), rode --help, preencha o protocolo-
pericia.yaml"
    fi
  done
done

echo "== Fim da triagem: nenhum item aqui foi aprovado automaticamente =="
```

Blindagem Extra: Grep de Segredo Antes do Commit

```
#!/usr/bin/env bash
# blindar-commit.sh -- grep de segurança antes de commitar o cofre de dotfiles.
# Uso: dentro do diretorio-fonte do chezmoi (apos `chezmoi cd`), antes do commit:
#   ./blindar-commit.sh

set -euo pipefail

PADROES_SEGREDO=(
  'sk-[a-zA-Z0-9_-]{20,}'
  '-----BEGIN [A-Z]*PRIVATE KEY-----'
  'AKIA[0-9A-Z]{16}'
)

echo "== Blindagem de segredo antes do commit =="
```

```
encontrou=0
for padrao in "${PADROES_SEGREDO[@]"; do
    if git diff --cached | grep -qE "$padrao"; then
        echo "BLOQUEADO: padrao de segredo encontrado no diff staged ($padrao)"
        encontrou=1
    fi
done

if [ "$encontrou" -eq 1 ]; then
    echo "Remova o segredo do arquivo e injete via 'op read' em tempo de uso."
    exit 1
fi

echo "Nenhum padrao de segredo encontrado. Commit liberado."
```

⑤ Verificação / Gate

```
sh -c "$(curl -fsLS https://get.chezmoi.io)"
```

⑥ Feito quando...

- ☐ Rode `--help` (ou equivalente) numa ferramenta de agente que você já usa antes de copiar qualquer comando novo de um tutorial está semana
- ☐ Classifique 3 fontes que você consulta no trabalho hoje como Classe A, B ou C, usando os critérios deste capítulo
- ☐ Preencha um `protocolo-perícia.yaml` para 1 comando que você copiou de algum lugar sem checar nos últimos 30 dias
- ☐ Rode `chezmoi init` num diretório de teste e confirme o hash SHA-256 do seu próprio `CLAUDE.md` antes e depois de uma edição
- ☐ Identifique, no seu ambiente de trabalho, algo que você trata hoje como “aprovado” mas que na verdade nunca teve fonte primária confirmando — reclassifique como pendente

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — O Manual Suspeito: Por Que Comandos Bonitos Enganam

- ☐ Escolha 1 comando de qualquer manual, tutorial ou post de blog que você usou nas últimas semanas sem checar a fonte oficial
- ☐ Rode `<comando> --help` (ou o equivalente da ferramenta) e compare linha a linha com o que você havia copiado
- ☐ Classifique o comando usando os 4 veredictos deste capítulo: CONFIRMADO, PARCIALMENTE CORRETO, FABRICADO ou NÃO VERIFICÁVEL
- ☐ Se for PARCIALMENTE CORRETO ou FABRICADO, escreva a versão corrigida ao lado da original, com a fonte primária anotada

Passo 2 — Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repeticao

- ☐ Localize o arquivo real de configuração do Claude Code na sua máquina (`~/claude/settings.json`) e liste os campos existentes — confirme que não há `cacheControl` nem `systemPrompt`
- ☐ Rode `npx ccusage@latest session --json` em um projeto com Claude Code já usado e identifique os valores de `cacheCreationTokens`, `cacheReadTokens` e `cacheHitRate`
- ☐ Se você usa Aider, adicione `cache-prompts: true` ao `~/aider.conf.yml` e rode duas chamadas seguidas sobre o mesmo arquivo para observar a diferença de custo
- ☐ Escreva, para o seu próprio `CLAUDE.md`, uma lista dos elementos que poderiam variar de uma execução para outra (data, hora, IDs aleatórios) e remova qualquer um deles do topo do arquivo

Passo 3 — Modelos Gratuitos Sem Lenda: o Catalogo Real Por Tras do Gateway

- ☐ Confiar em tabelas de benchmark encontradas em posts sem link para a medição original — trate como não verificado até achar a fonte primária
- ☐ Presumir que uma convenção de nome “bonita” (tiny/small/medium) é sinal de autenticidade — muitas vezes é o oposto
- ☐ Automatizar a escolha de modelo sem registrar a data da última checagem de catálogo — sem isso, você não sabe quando reconferir

Passo 4 — Compressao e Cache Semantico Sob a Lupa: Bibliotecas, Nao Comandos

- ☐ Rode `pip show llmlingua` no seu ambiente e confirme que não aparece nenhum “Console Scripts” na saída
- ☐ Escreva e teste `compress_prompt.py` com um prompt real de pelo menos 500 tokens, comparando o texto antes e depois
- ☐ Inicialize `cache.init()` do GPTCache localmente e force um cache hit repetindo a mesma pergunta duas vezes seguidas
- ☐ Ajuste `similarity_threshold` para 0.9 e depois para 0.6 e registre a diferença de comportamento em duas perguntas parecidas, mas não idênticas
- ☐ Suba `gptcache_server -s 127.0.0.1 -p 8000` localmente e confirme, consultando a doc oficial [7], o schema de payload real dos endpoints `/put` e `/get` antes de integrá-lo a qualquer script

Passo 5 — Paralelismo e Resiliencia Que Sempre Funcionaram

- ☐ Imagine a cena: você acabou de terminar o Capítulo 4

- ☐ Você escreve rápido um laço com `asyncio.gather` chamando as 80 revisões de uma vez
- ☐ Você roda o script e
- ☐ O diagnóstico é o mesmo problema que a seção Explica descreveu: `asyncio.gather` sozinho não impõe limite de concorrência nenhum — ele dispara todas as corrotinas fornecidas simultaneamente
- ☐ A correção tem duas partes: primeiro
- ☐ Meça a taxa de erro com os subcomandos reais de auditoria de uso que o Capítulo 6 destrincha a partir da ferramenta de contagem de tokens [21]
- ☐ Como síntese rápida das armadilhas mais comuns nesta frente: (1) esquecer o semáforo

Passo 6 — Orcamento e Fallback: Medir Antes de Limitar

- ☐ Imagine a cena: são 16h de uma sexta-feira
- ☐ O comando `pipx install ccusage` falha silenciosamente para você — na verdade
- ☐ Você gasta vinte minutos reinstalando `pipx`
- ☐ O diagnóstico correto seria outro: `ccusage` nunca foi um pacote Python
- ☐ É uma ferramenta Node — o próprio nome do ecossistema já era a evidência que faltava conferir antes de tentar instalar
- ☐ A correção é trivial depois que o laudo é refeito: `npx ccusage@latest daily --json` roda sem instalação alguma
- ☐ O erro comum não foi de sintaxe de shell — foi de não confirmar

Passo 7 — O Agente Avançado e as Outras Sete Ferramentas: Produto Real, Sintaxe Fabricada

- ☐ Rode `hermes skills --help`, `hermes cron --help`, `hermes memory --help` e `hermes sessions --help` na sua instalação e confira se os subcomandos batem com o laudo deste capítulo
- ☐ Execute `hermes_pericia.sh` e adicione uma linha nova para o próximo comando do Hermes que você pretende automatizar, antes de escrevê-lo em produção
- ☐ Escolha uma das sete ferramentas da tabela de campo (a que você já usa ou pretende testar) e confirme o caminho real de configuração na documentação oficial dela, registrando o resultado
- ☐ Rode `verificar_cli.py` contra uma CLI Agêntica qualquer que você já tenha instalada, com uma lista de 3 a 5 subcomandos que você pretende automatizar

Passo 8 — O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo

- ☐ Rode `--help` (ou equivalente) numa ferramenta de agente que você já usa antes de copiar qualquer comando novo de um tutorial está semana
- ☐ Classifique 3 fontes que você consulta no trabalho hoje como Classe A, B ou C, usando os critérios deste capítulo
- ☐ Preencha um `protocolo-pericia.yaml` para 1 comando que você copiou de algum lugar sem checar nos últimos 30 dias
- ☐ Rode `chezmoi init` num diretório de teste e confirme o hash SHA-256 do seu próprio `CLAUDE.md` antes e depois de uma edição
- ☐ Identifique, no seu ambiente de trabalho, algo que você trata hoje como “aprovado” mas que na verdade nunca teve fonte primária confirmando — reclassifique como pendente